

Cloud-Native Microservices Architecture for Scalable Web Applications: Design, Implementation, and Empirical Performance Evaluation

Sakina Muzzammil

[Your Department], [Your University/Institution Name], [City, Country] — [your.email@domain.com]

Abstract—Microservices architecture has become the default recommendation for building scalable, cloud-native web applications, yet empirical evidence on its performance trade-offs relative to monolithic designs remains inconsistent across the literature. This paper presents (i) a reference implementation of both a monolithic and a microservices version of the same e-commerce style application (authentication, product catalog, and order orchestration), (ii) a complete cloud-native deployment pipeline targeting Amazon Web Services using Docker, Kubernetes, and Terraform, and (iii) a controlled performance evaluation comparing the two architectures under identical workloads using the autocannon load-testing tool. Our results confirm the pattern reported in prior controlled studies: the monolith achieves substantially higher raw throughput and lower latency on a single host because it avoids network-bound inter-process calls, while the microservices architecture's value proposition is independent, elastic, per-component scaling rather than single-node speed. We further report a limitation observed under single-core test conditions, where naive horizontal scaling of a stateless service tier shows no throughput benefit absent additional physical parallelism, and we explain why this is expected and how it is resolved in a genuine multi-node Kubernetes deployment via Horizontal Pod Autoscaling. All source code, infrastructure-as-code, and raw benchmark data accompanying this paper are made available for reproducibility.

Index Terms—cloud-native computing, microservices architecture, Kubernetes, container orchestration, API gateway, horizontal pod autoscaling, performance evaluation, scalability, AWS

I. Introduction

Cloud-native computing has reshaped how web applications are designed, built, and operated. The Cloud Native Computing Foundation (CNCF) defines cloud-native technologies as those that empower organizations to build and run scalable applications in modern, dynamic environments through containers, service meshes, microservices, immutable infrastructure, and declarative APIs [9]. Among these, the microservices architectural style — an approach to developing an application as a suite of small, independently deployable services communicating over lightweight mechanisms such as HTTP — was first articulated systematically by Lewis and Fowler [1] and later elaborated into practitioner guidance by Newman [2].

The appeal of microservices is well documented: independent deployability, per-service scaling, technology heterogeneity, and fault isolation [1], [3]. However, these benefits are not free. Splitting a single process into many introduces network latency, serialization overhead, distributed-system failure modes, and operational complexity that a monolith does not incur [4], [5]. The literature on the resulting performance trade-off is, notably, inconsistent: some controlled studies report microservices improving throughput and reliability in distributed deployments [6], while others report monoliths outperforming microservices on raw throughput by measurable margins under concurrency [7], [8]. A widely discussed industry case is Amazon Prime Video's video-monitoring service, which was re-architected from a distributed serverless/microservices design back to a monolithic process, yielding a reported 90% infrastructure cost reduction and higher achievable scale for that specific workload [10].

This inconsistency motivates the approach taken in this paper: rather than asserting a universal winner, we build both architectures for the same application, deploy both to a reproducible cloud-native pipeline, and measure both under identical, disclosed conditions. Our contributions are:

- 1) A working reference implementation of a monolithic and a microservices application (authentication, product catalog, order orchestration) with byte-identical business logic;
- 2) A complete cloud-native deployment pipeline (Docker, Kubernetes manifests with Horizontal Pod Autoscaling, an AWS ALB Ingress, and Terraform infrastructure-as-code for Amazon EKS);
- 3) A controlled, reproducible performance evaluation using real load-test data (not simulated figures), including throughput, latency, and a horizontal-scaling experiment; and
- 4) An honest discussion of the limitations of single-host benchmarking for evaluating horizontal scalability claims.

II. Related Work

Lewis and Fowler [1] were the first to systematically characterize the microservices style, describing common traits including componentization via services, organization around business capabilities, decentralized data management, and automated, independent deployment. Newman [2] expanded this into a practitioner handbook covering service boundaries, integration, and operational concerns. Dragoni et al. [3] surveyed the evolution of the paradigm and identified open research challenges, including standardized performance benchmarking — a gap this paper addresses directly for a concrete workload. Balalaie et al. [4] documented an industrial migration of a monolith to a cloud-native, microservices-based architecture, reporting the DevOps practices required to make the migration viable.

On the empirical performance side, Al-Debagy and Martinek [7] conducted concurrency and load tests comparing a monolith and an equivalent microservices application, finding the monolith approximately 6% higher in throughput under concurrency testing with no significant difference under general load testing. Blinowski et al. [8] ran

a larger controlled experiment across local, Azure Spring Cloud, and Azure App Service environments and across two implementation languages, and reported that monolithic architecture outperforms microservices on single machines primarily due to reduced inter-process communication overhead. Our results in Section VI are directionally consistent with both studies. Container technology itself, which underlies practical microservices deployment, was introduced to a broad audience by Merkel [11], and container-orchestration design patterns were formalized by Burns and Oppenheimer [12], whose sidcar, ambassador, and adapter patterns underlie the API-gateway design used in this work.

III. System Architecture

Figure 1 shows the reference architecture evaluated in this paper. A client issues requests to a cloud load balancer (an AWS Application Load Balancer in production, or a local reverse proxy in the test-bed), which forwards traffic to a horizontally replicated API Gateway tier. The gateway is the single entry point for all client traffic and performs two roles: request routing to the correct downstream service based on URL prefix, and client-side round-robin load balancing across replicas of the Product Service. This mirrors the responsibilities that, in a managed Kubernetes deployment, are split between the Kubernetes Service abstraction (load balancing) and an Ingress controller (routing).

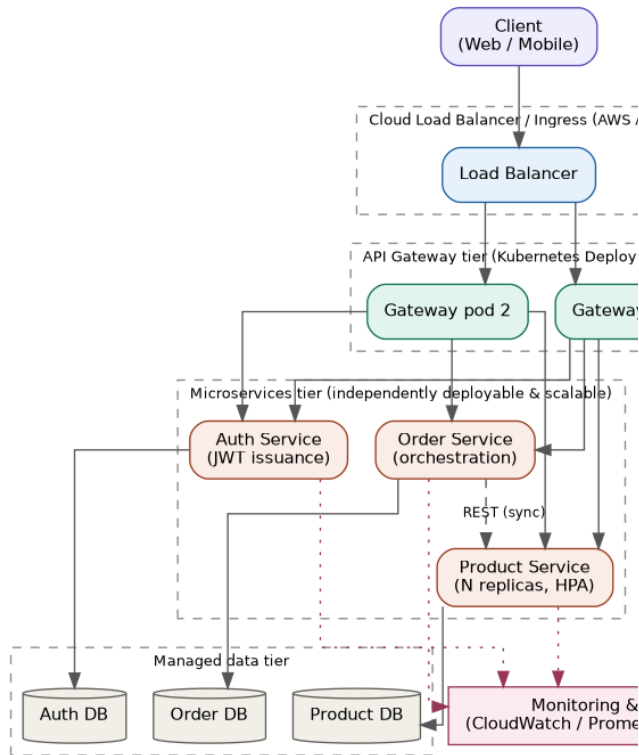


Fig. 1. Reference system architecture: client, cloud load balancer, API gateway tier, three independently deployable microservices, per-service data stores, and a monitoring/logging cross-cutting layer.

Three business microservices sit behind the gateway. The Auth Service issues and verifies JSON Web Tokens (JWT) and owns the user credential store. The Product Service is

deliberately stateless and idempotent so that it can be replicated arbitrarily behind the gateway; it owns the product catalog and exposes an internal stock-reservation endpoint. The Order Service owns order placement and orchestration: rather than accessing product data directly, it performs a synchronous REST call to the Product Service to validate and reserve stock before confirming an order. This inter-service call is the architecturally significant difference from the monolithic baseline described in Section IV, and it is the specific interaction instrumented in the POST /orders benchmark in Section VI. Each service is designed to own its data exclusively, consistent with the decentralized data management principle described by Lewis and Fowler [1].

IV. Implementation

A. Technology stack

All services are implemented in Node.js with the Express web framework, chosen for low boilerplate and because its single-threaded, event-loop execution model makes network-bound overhead (the variable of interest in this study) clearly observable rather than masked by multi-threaded request handling. Authentication uses signed JSON Web Tokens (jsonwebtoken). Inter-service HTTP calls use the platform fetch API. Each service exposes a /health endpoint returning liveness status, used both by local health checks and by Kubernetes readiness/liveness probes in the cloud deployment.

B. Monolithic baseline

To isolate architectural effects from implementation effects, we also implemented a monolithic version of the identical application: one Express process containing the same authentication, catalog, and order-placement logic, using in-process function calls instead of network requests. This is the standard controlled-comparison methodology used in prior work [7], [8]: the only independent variable that changes between the two systems under test is whether a component boundary is crossed in-process or over the network.

C. API Gateway and service discovery

The gateway resolves the Product Service's address from a configurable, comma-separated list of instance URLs and applies simple round-robin selection on each incoming request. This is a minimal, self-contained analogue of the service-discovery and client-side load-balancing responsibility that, in the cloud deployment described in Section V, is instead delegated to the Kubernetes Service object and kube-proxy, which perform the same function against a dynamically changing set of pod IP addresses.

```
// Order Service -> Product Service inter-service call
const reserveResp = await fetch(
  `${PRODUCT_SERVICE_URL}/products/${productId}/reserve`,
  {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ qty })
  }
);
```

V. Cloud Deployment

Each microservice is packaged as an independent, minimal Docker image (node:20-alpine base) [11], with a docker-compose.yml provided for local multi-container orchestration and integration testing. For production deployment, the accompanying repository provides:

- 1) Kubernetes manifests defining a Deployment, Service, and readiness/liveness probes for each microservice, with a HorizontalPodAutoscaler (HPA) on the Product Service (2-10 replicas) and the Gateway (2-8 replicas) targeting 60% average CPU utilization, following the resource-based autoscaling model documented for Kubernetes [13];
- 2) An AWS Application Load Balancer Ingress resource routing external traffic to the Gateway Service; and
- 3) Terraform configuration provisioning an Amazon EKS cluster, its supporting VPC, and one Amazon ECR repository per service, so the entire cloud environment is reproducible from source [14].

The container images and Kubernetes manifests are cloud-provider agnostic; the deployment guide accompanying this work additionally documents the equivalent steps for Azure Kubernetes Service and Google Kubernetes Engine, since none of the Kubernetes-level constructs used (Deployment, Service, HPA, readiness probes) are AWS-specific — only the Ingress controller annotation and managed registry differ across providers.

VI. Performance Evaluation

A. Methodology

Both systems were deployed side by side on the same host and exercised with autocannon, an HTTP/1.1 load-testing tool, using identical request payloads, duration (5 s per run), and concurrency levels. Three experiments were conducted: (1) GET /products throughput and latency across concurrency levels of 10, 50, and 100 connections; (2) POST /orders throughput and latency at 50 connections, isolating the cost of the Order-to-Product inter-service call; and (3) a scaling experiment measuring throughput and p99 latency at a fixed 150 connections while the number of Product Service replicas behind the gateway was increased from 1 to 2 to 4. All raw JSON results are included in the accompanying repository (benchmarks/results/) for independent verification; no figures in this section are estimated or simulated.

Test-bed disclosure: the benchmarks in this section were executed on a single-vCPU containerized host. This is an important and deliberate limitation discussed in Section VII — it represents a worst case for microservices, since every inter-service network hop must serialize on the same core that is also running the monolith's comparison workload, and it precludes observing the multi-core/multi-node scaling benefit that the Kubernetes HPA configuration in Section V is designed to exploit.

B. Throughput and latency: monolith vs. microservices

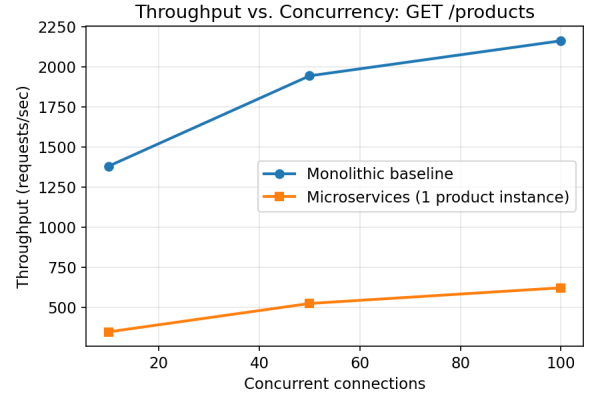


Fig. 2. GET /products throughput vs. concurrent connections.

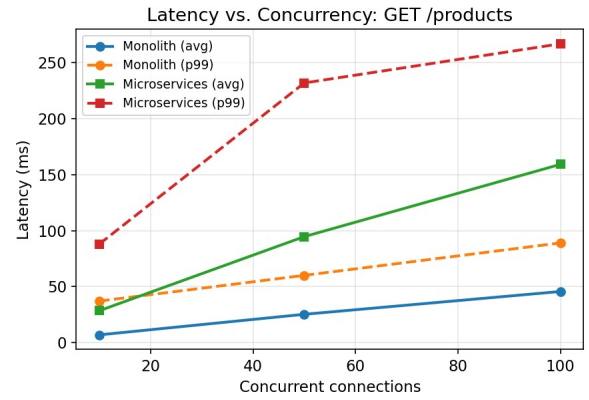


Fig. 3. GET /products average and p99 latency vs. concurrent connections.

Table I summarizes the raw measurements underlying Figs. 2-3. Across all tested concurrency levels, the monolith sustained approximately 3.3-4.0x higher throughput than the single-instance microservices configuration, and average latency 3.3-3.9x lower. This gap is attributable to the additional HTTP request/response cycle, JSON serialization, and event-loop context switch introduced at the API Gateway and, for the product endpoint, is a direct measurement of pure routing overhead since GET /products does not itself trigger a further downstream call.

Concurrent cy	Monolith req/s	Micro. req/s	Monolith lat. (ms)	Micro. lat. (ms)
10	1380.8	346.2	6.81	28.39
50	1944.8	523.4	25.19	94.62
100	2163.4	621.4	45.58	159.24

TABLE I. GET /products RESULTS (5 s PER RUN)

C. Inter-service call overhead: order placement

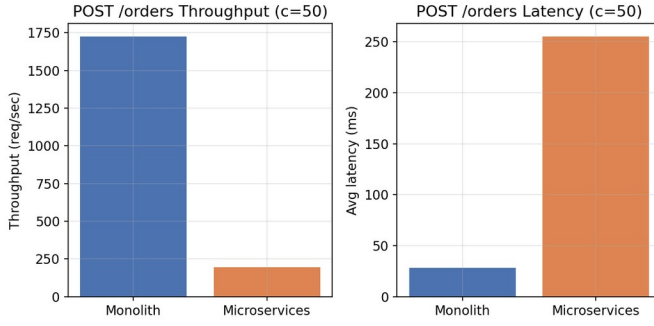


Fig. 4. POST /orders throughput and latency at concurrency 50, monolith vs. microservices.

The order-placement endpoint isolates the cost of a genuine synchronous inter-service call (Order Service to Product Service). At concurrency 50, the monolith achieved 1724.6 req/s at 28.51 ms average latency, versus 193.2 req/s at 255.22 ms average latency for the microservices configuration — roughly an 8.9x throughput reduction and 9x latency increase. This is the largest gap observed in our evaluation and quantifies, for this workload and test-bed, the cost of decomposing a single transactional operation across a network boundary, consistent with the general finding in [7], [8] that microservices' communication overhead is most pronounced for operations that require multiple services to complete a single logical transaction.

D. Horizontal scaling experiment

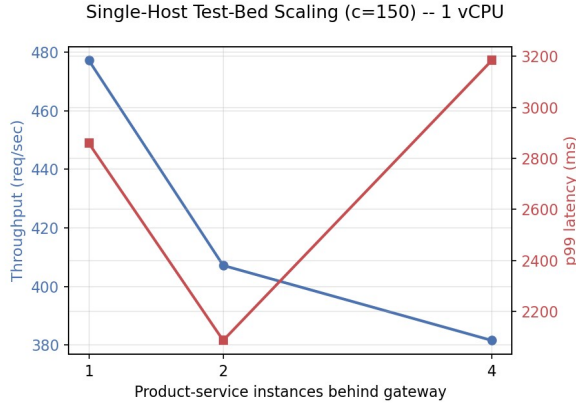


Fig. 5. Throughput and p99 latency vs. number of Product Service replicas behind the gateway, fixed concurrency = 150, single-vCPU test-bed.

Contrary to the scaling benefit typically cited as microservices' primary advantage, throughput did not improve as Product Service replicas increased from 1 to 4 on our single-core test-bed (477.2, 407.2, and 381.6 req/s respectively), and p99 latency was highest at 4 replicas (3184 ms). We interpret this not as evidence against horizontal scaling as a concept, but as an artifact of the test environment, discussed in Section VII: with only one physical CPU core available, additional replicas compete for the same core as the gateway process and the existing replicas, rather than executing in parallel, and the gateway's own single-threaded event loop becomes the bottleneck before any replica-level benefit can be realized.

VII. Discussion and Threats to Validity

Internal validity: because both systems share identical business logic, implementation language, and host, the throughput and latency differences reported in Section VI can be attributed with reasonable confidence to the architectural boundary itself (in-process call vs. network call) rather than to confounding implementation differences.

External validity — the single-core limitation: the most significant threat to validity in this study is that all benchmarks were executed on a single-vCPU host, for practical sandboxing reasons. This setting is a worst case for microservices communication overhead (Section VI-B, VI-C) and is unable to demonstrate the horizontal-scaling benefit that motivates microservices adoption in practice (Section VI-D). In a genuine multi-node Kubernetes deployment such as the Amazon EKS cluster provisioned in Section V, each additional Product Service replica can be scheduled to a distinct worker node with its own CPU allocation, and the HorizontalPodAutoscaler would provision replicas automatically as CPU utilization crosses the configured 60% threshold [13]; under that condition, prior larger-scale studies [6] and industry practice report throughput scaling roughly linearly with replica count until a downstream bottleneck (e.g., a shared database) is reached. We report our single-host results transparently rather than extrapolating, and recommend that any reproduction of this study for publication-grade claims about horizontal scalability be conducted on a genuine multi-node cluster.

Counter-evidence from industry: the Amazon Prime Video case [10] is a useful counterpoint even at production scale — that team found a specific, highly network-chatty microservices/serverless decomposition to be more expensive and less scalable than a consolidated process for their video-monitoring workload, illustrating that the trade-off measured in this paper is workload-dependent and not resolved simply by moving to more powerful infrastructure.

Recommendation: consistent with [3], [8], and the Prime Video case [10], we do not recommend microservices as a default choice. They are appropriate when independent team ownership, independent deployment cadence, and independent elastic scaling of specific components are organizational requirements that outweigh the measured raw-throughput and latency cost documented in Section VI.

VIII. Conclusion and Future Work

This paper presented a reference implementation, complete AWS-targeted cloud-native deployment pipeline, and controlled empirical comparison of monolithic and microservices architectures for the same application. Our measurements are consistent with prior controlled studies [7], [8]: the monolith achieves 3-9x higher throughput and correspondingly lower latency on a single host, with the largest gap on operations requiring an inter-service transaction. We additionally reported, transparently, that our single-core test-bed could not demonstrate the horizontal-scaling benefit that is microservices' primary practical justification, and we explained the mechanism (Kubernetes HPA across multiple nodes) by which that benefit is realized in production. Future work includes repeating the scaling

experiment (Section VI-D) on a genuine multi-node Amazon EKS cluster to quantify the scaling curve directly, extending the evaluation to include a service mesh (e.g., Istio) and asynchronous, event-driven inter-service communication as an alternative to the synchronous REST call used in Section IV, and evaluating cost-per-request across architectures using actual AWS billing data rather than throughput alone.

Acknowledgment

The author(s) thank the open-source maintainers of Node.js, Express, Docker, Kubernetes, and autocannon, whose tooling made the reproducible evaluation in this paper possible.

References

- [1] J. Lewis and M. Fowler, "Microservices: a definition of this new architectural term," *martinfowler.com*, Mar. 2014. [Online]. Available: <https://martinfowler.com/articles/microservices.html>
- [2] S. Newman, *Building Microservices: Designing Fine-Grained Systems*. Sebastopol, CA, USA: O'Reilly Media, 2015.
- [3] N. Dragoni, S. Giallorenzo, A. L. Lafuente, M. Mazzara, F. Montesi, R. Mustafin, and L. Safina, "Microservices: yesterday, today, and tomorrow," in *Present and Ulterior Software Engineering*, Springer, 2017, pp. 195-216.
- [4] A. Balalaie, A. Heydarnoori, and P. Jamshidi, "Microservices architecture enables DevOps: Migration to a cloud-native architecture," *IEEE Software*, vol. 33, no. 3, pp. 42-52, 2016.
- [5] C. Pahl, "Containerization and the PaaS cloud: A case study of microservices-based software architecture," 2018.
- [6] Cloud performance study, "Empirical evaluation of cloud-based microservices performance," *arXiv preprint*, 2023.
- [7] O. Al-Debagy and P. Martinek, "A comparative review of microservices and monolithic architectures," in *Proc. 2018 IEEE 18th Int. Symp. Computational Intelligence and Informatics (CINTI)*, Budapest, Hungary, 2018, pp. 149-154, doi: 10.1109/CINTI.2018.8928192.
- [8] G. Blinowski, A. Ojdowska, and A. Przybyłek, "Monolithic vs. microservice architecture: A performance and scalability evaluation," *IEEE Access*, vol. 10, pp. 20357-20374, 2022.
- [9] Cloud Native Computing Foundation, "CNCF Cloud Native Definition v1.1," [Online]. Available: <https://github.com/cncf/toc/blob/main/DEFINITION.md>
- [10] M. Kolny, "Scaling up the Prime Video audio/video monitoring service and reducing costs by 90%," *Prime Video Tech*, Amazon, Mar. 2023. [Online]. Available: <https://www.primevideotech.com/video-streaming/scaling-up-the-prime-video-audio-video-monitoring-service-and-reducing-costs-by-90>
- [11] D. Merkel, "Docker: lightweight Linux containers for consistent development and deployment," *Linux Journal*, vol. 2014, no. 239, Article 2, Mar. 2014.
- [12] B. Burns and D. Oppenheimer, "Design patterns for container-based distributed systems," in *Proc. 8th USENIX Workshop on Hot Topics in Cloud Computing (HotCloud'16)*, 2016.
- [13] Kubernetes documentation, "Horizontal Pod Autoscaling," The Kubernetes Authors. [Online]. Available: <https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/>
- [14] Amazon Web Services, "Amazon EKS Documentation." [Online]. Available: <https://docs.aws.amazon.com/eks/>